Unit: a small dependent type checker with segments as modules

19 October 2026

Abstract

Unit is a prototype proof checker for a dependent type theory with Π, Σ, records, labelled sums, an identity type with its eliminator, and a hierarchy of cumulative universes indexed by level expressions. Its module system is based on the segments of Automath, in the form given in [4]: a module is a list of declarations, its parameters are the declarations without a body, and an instance only binds the parameters, without copying the body. Records are declared apart from modules, with the same syntax for their bodies: a record type has fields, one constructor and an eliminator, and its elements are compared by their fields, without η. The prototype has been used to formalise a model of univalent type theory over posets and simplicial sets, about 200 files. This note describes the type theory, the module system, and the choices made to keep checking predictable and fast. The name Unit refers to the units, the declarations of which every file, segment and local block is a list.

Contents

1 Overview **2**

2 The type theory **2**
2.1 Terms . . . 2
2.2 Universes and levels . . . 3
2.3 Definitional equality . . . 4

3 The module system **5**
3.1 Segments . . . 5
3.2 Instances share . . . 6
3.3 Parameters stated once . . . 6
3.4 Records . . . 7

4 The checker **7**

5 Notation read before checking **8**
5.1 Omitted arguments . . . 8
5.2 Equational chains . . . 9
5.3 Scoping . . . 9

1 Overview

A file is a list of *units*. A unit is a declaration of one of the following kinds:

`x (tel) : A ;`	a parameter (at the top level: a primitive notion)
`x (tel) : A = e ;`	a definition
`data D (tel) = c1 (tel1) | ...;`	a labelled sum (a definition)
`level a b ;`	level variables
`constraint a < b ;`	constraints between levels
`module M (tel) { units } ;`	a segment
`module N = M[args] ;`	an instance of a segment
`record R (tel) { units } ;`	a record type
`opaque f ; transparent f ;`	the unfolding strategy

(The scoping declarations `open` and `import` are read before checking; see Section 5.) Checking a list of units threads a *context of units*: each unit is checked in the context of the units before it, and is then added to it. A definition stays a definition in the context: its body is available for computation. The body of a `let` is a list of units as well. The scope is sequential: a definition sees only the units before it, never itself or a later one, so ordinary definitions cannot form cycles. Recursion comes only from the eliminators of labelled sums (Section 2).

Two design principles run through the system. First, *nothing is guessed*: there is no unification, and the few arguments that may be omitted are found by a deterministic matching procedure (Section 5.1). Second, *nothing is copied*: a definition has one identity wherever it is used, and two uses are compared by that identity before anything is unfolded (Section 2.3).

2 The type theory

2.1 Terms

The terms are those of a dependent type theory with definitions:

`U l`	the universe at level `l`
`(x : A) -> B, \ x -> t, t u`	dependent functions
`(x : A) * B, (t, u), t.1, t.2`	dependent pairs
`R[args], new R[args], t.l, split { R x1...-> t }`	records
`sig { l1 : A1 ; ...}, struct { l1 = t1 ; ...}`	anonymous records
`c t1 ...tn, split { c1 x1...-> t1 ; ...}`	constructors and case analysis
`Id A a b, refl a, idJ A a C d b p`	the identity type
`[a] A, <a> t, t @l`	products over levels
`[a < b] A`	products over constraints
`let { units } in t`	local definitions
`M[args].x`	a member of a segment instance

Labelled sums. A `data` declaration introduces a type with named constructors, each with a telescope of arguments. A function out of a sum is written `split`, a list of branches, one per constructor; `split` is not a term on its own but is checked against a function type $(x : D) \to P\,x$ whose domain is a sum. Recursion is not by name: `Unit` has *primitive recursion*, through the eliminators of the sums. A branch may bind, after its pattern variables, one *recursive result* for each recursive field of the constructor:

```
data N = zero | suc (n : N) ;
plus : N -> N -> N =
  split { zero -> \ n -> n ; suc m | r -> \ n -> suc (r n) } ;
```

For a field $m : D$ the result has type $P\,m$; for a function field $f : (y : \Psi) \to D$, as in `sup (A : U l) (f : A -> M)`, it has type $(y : \Psi) \to P\,(f\,y)$. The motive P contains everything after the argument split, so the result is a function when the split is followed by further arguments, as $r : N \to N$ above. The arguments before the split are fixed. A nested split has its own results: in Ackermann's function

```
ack : N -> N -> N = split {
  zero -> \ n -> suc n ;
  suc m | r -> split { zero -> r (suc zero) ; suc n | s -> r s } } ;
```

`s` is `ack (suc m) n` and `r` is `ack m`. This is primitive recursion at higher types, as in Gödel's system T: since a recursive result may be a function, Ackermann's function is definable. Mutual recursion is written as one recursion computing a pair, and course-of-values recursion likewise. The list of results may be omitted when a branch does not use them; when present it is complete, and `_` names an unused result.

The sums are restricted so that these eliminators are the usual ones. A data type sees its own name only in its constructor fields, never in its parameters or type, so there are no mutual sums. A field mentions the type only as $D\,\vec{p}$ or $(y : \Psi) \to D\,\vec{p}$, where Ψ does not mention D and $\vec{p}$ are exactly the parameters of D: the sums are strictly positive, with uniform parameters, and there are no nested or indexed inductive families. Every recursive computation then starts from an eliminator whose recursive results the checker constructs, on a recursive field of a constructor value: every recursion is structural.

A `split` or a `data` may occur only at a *definition position*: as the body of a definition, after its λs, or inside the branches of such a `split`. Elsewhere, for instance as the argument of a function, it is rejected. A `split` is then written back as its definition applied to the values of the variables on its path, a term in the current scope, provided its recursive results are those that path determines; otherwise (for instance under the abstract results of a branch being compared) it is not written back.

The identity type. `Id A a b` is a primitive type with `refl a : Id A a a` and the eliminator

$$\frac{C : (y : A) \to \mathsf{Id}\,A\,a\,y \to \mathsf{U}\,l \qquad d : C\,a\,(\mathsf{refl}\,a) \qquad p : \mathsf{Id}\,A\,a\,b}{\mathsf{idJ}\,A\,a\,C\,d\,b\,p : C\,b\,p}$$

with the computation rule $\mathsf{idJ}\,A\,a\,C\,d\,a\,(\mathsf{refl}\,a) = d$. Since there is no pattern matching on `refl`, there is no way to derive uniqueness of identity proofs. With strictly positive sums and recursion only through their eliminators, the theory is meant to be compatible with univalence, which a development may assume as a parameter (Section 3.3).

2.2 Universes and levels

The universes follow the system of [2]. Levels are expressions

$$l \;::=\; a \;\mid\; l^{+} \;\mid\; l \vee m$$

in level variables a, a successor and a maximum. There is no level 0 for the user. Level variables are declared by `level a`, and may be constrained by `constraint` units ($l < m$, $l \leq m$, $l = m$).

The universes are *à la Russell*: an element of `U l` is itself a type. The rules are

- $\mathsf{U}\,l : \mathsf{U}\,l^+$;
- Π, Σ, records and Id lie in the maximum of the levels of their components;
- cumulativity: if $A : \mathsf{U}\,l$ and $l \leq m$ then $A : \mathsf{U}\,m$. Cumulativity applies to the whole type and not under a binder: a function into $\mathsf{U}\,l$ is not silently a function into $\mathsf{U}\,m$, and one writes $\lambda b.\, T\,b$ when needed;
- a sum declared without a universe, like `N` above, is *small*: it lies in every universe. A sum may also be declared at a given level, `data D : U l = ...`.

Products over levels and constraints. A type may be polymorphic in a level: `[a] A` is the type of the `<a> t`, applied to a level by `t @l`. A type may also depend on a constraint: `[a < b] A`. A constraint product is introduced and eliminated silently: a term of type $[\psi]A$ is used as a term of type A wherever ψ holds in the current context. Both kinds of products are *large*: they lie in no universe.

Deciding constraints. Whether a constraint follows from those of the context is decided by the algorithm of Bezem and Coquand for join-semilattices with an inflationary endomorphism: constraints become Horn clauses, and the question is answered by computing a least model by forward reasoning [1]. The same algorithm detects *loops*, contexts whose constraints force $l < l$. A context with a loop is inconsistent, and in it every two terms are convertible. In practice this signals an error in the constraints, and the checker warns.

2.3 Definitional equality

Definitional equality is generated by

- β for functions, pairs, level abstractions, projections and `split` applied to a constructor (of a sum or of a record); for a sum, the recursive results of the branch are the same `split` applied to the recursive fields;
- the computation rule of `idJ` on `refl`;
- the unfolding of definitions (but see `opaque` below);
- η for functions and pairs.

There is no η for records: an element `t` of a record type and `new R[ps, t.x`$_1$`, ...]` are not convertible. They are equal in Id, by the eliminator (Section 3.4).

The checker tests it as follows (a time budget remains as a safeguard, Section 4). Two values are first compared *structurally*, without any computation. A use of a definition is represented by its name together with its environment: the values of all the variables and levels in scope where it is declared (its *arguments*), not only those its body uses. Two uses of the same definition with convertible arguments are equal without unfolding. Only when this fails are both sides brought to head normal form, unfolding definitions, and compared component by component.

Two occurrences of the same `split` are compared in the same way, by their environments. When these differ, the checker compares the branches instead, with fresh variables for the pattern variables and for the recursive results. This rule is kept as an experiment: it is counted, and its justification, which must cover dependent motives and the fresh recursive results, is still to be written.

Taking the whole environment is a deliberate choice: it is simple and needs no analysis of the bodies. Its consequence is visible under `opaque`: an opaque definition is never unfolded, so two uses of it that differ in a variable its body does not use are different.

Record types are compared by name. Two record types are equal when they come from the same declaration with equal parameters; their fields are never compared. Two different declarations are different types even if their fields agree. This is the rule of the proof checker HALF [3] and of Gofer, and it avoids comparing large records field by field. An anonymous `sig{...}` is a record declared at its occurrence. Dependent pairs $(x : A) * B$ are compared structurally. Elements of a record type are compared by their constructor: two elements built by `new` are equal when their parameters and fields are convertible, wherever they are written.

The unfolding strategy. `opaque f` declares that, from this point on, `f` is not unfolded: it behaves as a constant, though its body has not changed. `transparent f` undoes this. The strategy is a property of the checking run and not of the values: a value is computed in the same way whatever the strategy, and only the comparison consults it. In particular an `opaque` declaration placed after a segment applies to every instance of it. Making a definition opaque can only make fewer terms convertible, so it never makes an incorrect proof correct. The intended use is to lock a construction right after the lemmas that characterise it, as one does in informal mathematics.

3 The module system

3.1 Segments

A segment is a named list of units:

```
module Mon (A : U a) (e : A) (op : A -> A -> A) {
  twice (x : A) : A = op x x ;
  four (x : A) : A = twice (twice x) ;
} ;
```

The *parameters* of a segment are its units without a body: the telescope in the header and the parameters declared in the body. The level variables and the constraints of its body belong to it as well. A segment is checked once, where it is declared, with its parameters as unknowns.

An *instance* `Mon[N, zero, plus]` gives values to the parameters, in order. Each argument is checked against the type of its parameter, in which the earlier arguments have been substituted; a level parameter takes a level; a constraint of the segment must hold for the given levels. Members are then accessed as `Mon[N, zero, plus].four`, or through an alias `module NM = Mon[N, zero, plus] ;` as `NM.four`. Arguments may also be given by name, `Mon[A := N, ...]`.

Segments nest. A segment declared inside another sees the parameters of the enclosing one lexically, and needs no arguments for them:

```
module Outer (A : U a) {
  module Inner { k (x : A) : A = x ; } ;
  kk (x : A) : A = Inner.k (Inner.k x) ;
} ;
```

A development spread over several files is a nest of segments: each file becomes a segment (Section 5), and a file included inside a segment sees the enclosing parameters.

3.2 Instances share

An instance does not copy the body of the segment. It only records the values of the parameters, in an environment; the members are evaluated in that environment when they are used. A member therefore keeps a single identity, its qualified name, and two occurrences of a member are compared by this name and the values of the parameters of its environment. In particular `Mon[N, zero, plus].twice` reached through two different aliases is one object, and the equation

```
share : Id (N -> N) (Mon[N, zero, plus].twice) (NM.twice) = refl NM.twice ;
```

holds without unfolding `twice`. When a large development instantiates the same segment many times, as a formalisation of a model typically does (a construction made over an abstract base, then applied to each instance), this is what keeps checking fast: equal things are recognised as equal by their names.

3.3 Parameters stated once

Because segments nest and share, assumptions of a whole development are stated once, as parameters of one outer segment. For instance the model formalised with Unit assumes function extensionality and univalence for the metatheory; the main file reads

```
module Model (fe : [b] Axioms[b].FunExt) (univ : [b] Axioms[b].Univalence) {
  import V ;
  import Presheaf ;
  ...
} ;
```

where each `import` inserts a file as a sub-segment (Section 5), so that every file sees `fe` and `univ`. Likewise the abstract base of the model (a category with an interval and a presheaf of cofibrations) is one segment whose parameters are the data of the base; the generic constructions are imported inside it, and an instance of the base (posets, or simplicial sets) is one instance of that segment, after which every generic construction is a member of the instance. Excluded middle, used at one place, is the parameter of a separate segment, so the files that may use it are exactly those imported inside that segment.

Changing the assumptions. Function extensionality follows from univalence. Once this is proved, as a segment `FunExtUA` producing, from univalence at a level b, function extensionality at b, the whole development is obtained under univalence alone by one instance:

```
module ModelUA (univ : [b] Axioms[b].Univalence) {
  feU : [b] Axioms[b].FunExt = <b> FunExtUA[b].FE[univ @b].fe ;
  module M = Model[feU, univ] ;
} ;
```

Every result of the development is then available as `ModelUA[univ].M.....` Since an instance only binds the parameters, checking it checks the two arguments and nothing of the body: the development is neither copied nor checked again.

3.4 Records

A segment is never a type. A record is declared like a segment, with the keyword `record`:

```
record Mon (l : Level) {
  A : U l ;  op : A -> A -> A ;
  sq (x : A) : A = op x x ;
  e : A ;  sqe : Id A (sq e) e ;
} ;
```

Its parameters are those of the header; the declarations of the body without a definition are its *fields*, and its definitions are its *derived members*. A field type may use the earlier fields and derived members.

- *Formation.* `Mon[a]` is a type: all parameters are given, the fields are not. It lies in the maximum of the universes of its fields.
- *Construction.* `new Mon[a, N, plus, zero, refl zero]`, with the parameters and then the fields, is an element; `new` is the only constructor.
- *Projection.* For `M : Mon[a]`, `M.op` is a field, and `(new Mon[a, N, plus, ...]).op` computes to `plus`.
- *Derived members.* `M.sq` is the definition `sq` applied to the element `M`. Like any definition, it unfolds unless it is `opaque`, on a constructed element and on a variable alike, and two uses are equal when they apply it to equal elements.
- *Elimination.* `split { Mon A op e sqe -> b }` binds the fields (the parameters are given by the type) and computes on an element built by `new`. It proves η:

```
etaM : (M : Mon[a]) -> Id (Mon[a]) M (new Mon[a, M.A, M.op, M.e, M.sqe]) =
  split { Mon A op e sqe -> refl (new Mon[a, A, op, e, sqe]) } ;
```

A record comes with a constructor `mkR`, the function of its parameters and fields returning `new R[...]`, defined right after it; it is not generated when a field type uses a derived member, as `sqe` does here. A record may be declared inside a segment and use its parameters; its identity then includes the values of these parameters. The body of a record may `open` an instance of a segment, which may depend on earlier fields: this only gives short names to the members of the instance, which keep their identities.

Fixed fields. `Mon[a, N]` or `Mon[l := a, A := N]` is the type of the elements of `Mon[a]` whose field `A` is `N`. For `M : Mon[a, N]`, a projection `M.A` is read as `N` where it is checked, and the types of the later projections use `N`: `M.e : N`. A field may be fixed only together with the fields its type mentions. Fixing more fields gives a subtype: `Mon[a, N]` is a subtype of `Mon[a]`. Since the fixed value comes from the type of the projection, it is lost when an element is used at the larger type: for `pass (M : Mon[a, N]) : Mon[a] = M`, `(pass M).A` is not convertible to `N`, though the two are equal in `Id`, by the eliminator.

4 The checker

The checker is written in Haskell, about 6000 lines, with no library beyond the base. Its parts are: levels and the decision of constraints; evaluation into values with closures; head normalisation

under the unfolding strategy; conversion; type checking of units; the filling of holes, which also records in each `split` the positions of the recursive fields (evaluation reads only this record); and the parser and scope resolution, which also enforces the sequential scope, checks that the sums are strictly positive with uniform parameters, and computes, for each definition, the variables it depends on.

The context after a unit does not depend on whether the unit checked: definitions enter the context by their types and bodies, not by their proofs of correctness. Hence the result of each declaration can be computed separately, with a time budget (ten seconds per declaration; an overrun counts as a failure), and one file can be checked while the others are trusted. The checker also runs as a server that loads a development once and, after an edit, checks only the files whose content changed and those depending on them; the 210 files of the model (about 12000 declarations) check from scratch in about 54 seconds, and each file separately, the others trusted, in under five seconds.

Moving to recursion through eliminators changed little in the model: of 174 recursive definitions, 161 were rewritten mechanically, a call `f m` becoming the recursive result for `m`; 12 helpers that were mutually recursive with their caller now take the recursive result as an argument; and one proof (equality of lifts of iterative sets) was restated as a recursion on one argument. Checking time did not change. A hidden argument whose filling failed, and a recursive result whose positions were not recorded, are errors when used, so that they make no equality in a run that trusts the definition.

Remark 1. What Unit does not have: no unification and no implicit arguments beyond the matching of Section 5.1; no indexed inductive families; no recursion by name (only primitive recursion, through the eliminators); no mutual, nested or indexed inductive types; no pattern matching on `refl`; no proof irrelevance. These omissions are deliberate: each would make checking less predictable, and the formalisation did not need them.

5 Notation read before checking

The following conveniences are not part of the core language: they are translated into core terms before checking, and the core never sees them.

5.1 Omitted arguments

A parameter written in braces, `f {x : A} (y : B) : C`, is *hidden*: the type of `f` is the ordinary $(x : A) \to (y : B) \to C$, and a use `f u` is read as `f _ u`, where `_` is a hole. A hole may also be written explicitly anywhere, and a hidden argument may be given explicitly as `{e}`. Holes are filled before the term is checked, by one-way *matching*, never by unification:

- the inferred type of each explicit argument is matched against the corresponding domain, with the holes as unknowns; two applications are matched argument by argument only when they have the same head; otherwise both are computed and matching continues on their head normal forms;
- the expected type only fills holes left open by the explicit arguments;
- two different solutions for one hole are an error, so the result does not depend on the order;
- a solution is written back as a term in the current scope, using names only.

A hole that is not determined this way is an error that asks for an explicit `{e}`. This works well for lemmas about paths, such as `transH p (symH q)`, whose endpoints are determined by the explicit arguments, and it is deliberately not clever.

5.2 Equational chains

The notation

```
calc A { a0 =< p1 > a1 =< p2 > a2 ... =< pn > an }
```

proves $\mathsf{Id}\, A\, a_0\, a_n$. It stands for the right-nested composite of the paths p_i; each p_i is checked against $\mathsf{Id}\, A\, a_{i-1}\, a_i$, so every point is written once and an error is reported at its step.

5.3 Scoping

`open M ;` (or `open M[args] ;`) makes the members of a segment available by their short names. The resolver replaces every short name by the qualified name it stands for, `M[args].x`, so the core only sees qualified names. For a variable `c` of a record type, `open c` gives short names to `c.x`. `import F ;` is expanded when the files are read: the file `F.seg` is inserted as `module F { ...} ;`, once. A `where` block is a `let`, sequential like every list of units: its definitions see the earlier ones and the parameters of the definition, but not the definition itself. The constructor `mkR` of a record (Section 3.4) is an ordinary definition added by the resolver after the record. The resolver also separates segments from records: a segment used as a type, `new` applied to a segment, or a member of a record reached without an element, is an error.

References

[1] M. Bezem and T. Coquand. Loop-checking and the uniform word problem for join-semilattices with an inflationary endomorphism. *Theoretical Computer Science* 913, 2022.

[2] M. Bezem, T. Coquand, P. Dybjer and M. Escardó. Type theory with explicit universe polymorphism. arXiv:2212.03284, 2022 (revised and extended version, 2024).

[3] J. Cederquist. *A Pointfree Approach to Constructive Analysis in Type Theory*. PhD thesis, Chalmers University of Technology and University of Göteborg, 1997. Section 7.1 describes the proof checker HALF.

[4] Q. Wang. *A Haskell Implementation for a Dependent Type Theory with Definitions*. Master's thesis, Chalmers University of Technology and University of Gothenburg, 2021.